Ritma — Proje ve Kullanım Kılavuzu

> Randevu ve Müşteri Takip Sistemi · Sürüm 0.1.0 · Ekim 2026

Bu belge üç soruya cevap verir:

1. **Proje nasıl yapıldı?** Kararlar, katmanlar ve geliştirme adımları (Bölüm 2)
2. **Nasıl kurulur ve çalıştırılır?** (Bölüm 3–4)
3. **Uygulama nasıl kullanılır?** Ekran ekran anlatım (Bölüm 5–6)

Bunların yanında yapılandırma, sorun giderme, bilinen sınırlamalar ve Git/GitHub kullanımı da vardır.

GitHub deposu: https://github.com/tgcksr/ritma

İçindekiler

1. Genel bakış
2. Proje nasıl yapıldı?
3. Kurulum
4. Çalıştırma
5. Kullanım kılavuzu (ekran ekran)
6. İlk gün senaryosu
7. Yapılandırma (.env)
8. Geliştirici komutları ve testler
9. Klasör yapısı
10. Sorun giderme
11. Bilinen sınırlamalar ve sonraki adımlar
12. Git ve GitHub

---

1. Genel bakış

Ritma, randevuyla çalışan işletmeler (kuaför, güzellik merkezi, klinik, danışmanlık vb.) için web tabanlı bir yönetim uygulamasıdır.

Modül	Ne yapar?
Giriş	Yalnızca yetkili kullanıcı girebilir; oturum 12 saat geçerlidir.
Özet (Dashboard)	Günlük/haftalık/aylık randevu yoğunluğu, tamamlanan işlemler, gerçekleşen ve beklenen gelir.
Takvim	Randevu oluşturma, düzenleme, sürükle-bırak ile taşıma, durum değiştirme. **Çakışan randevuları engeller.**
Müşteriler (CRM)	Ekleme, düzenleme, silme, arama; profil sayfasında tüm randevu ve işlem geçmişi.
Hizmetler	Hizmet adı, süresi, ücreti; aktif/pasif durumu.
Personel	Personel bilgileri ve her personelin verdiği hizmetler. Randevuda personel seçimi isteğe bağlıdır.
Bildirimler	Randevudan 24 saat ve 1 saat önce SMS/e-posta hatırlatması (şu an simülasyon).
Dil	Arayüz Türkçe veya İngilizce; seçim tarayıcıda hatırlanır.

2. Proje nasıl yapıldı?

2.1 Teknoloji seçimi ve gerekçeleri

Katman	Teknoloji	Neden seçildi
Dil	**TypeScript** (her yerde)	Önyüz ve arkayüz aynı dilde olur; tipler hataları derleme aşamasında yakalar.
Arkayüz (API)	**Node.js 24 + Fastify 5**	Hafif ve hızlı; eklenti sistemi temiz.
Veritabanı erişimi	**Prisma 6**	Tip güvenli sorgular, migration (şema geçmişi) desteği, SQL injection'a karşı güvenli.
Veritabanı	**SQLite** (geliştirme) → **PostgreSQL** (canlı)	Kurulum gerektirmeden başlanır; geçiş tek satırlık ayardır.
Doğrulama	**Zod** (ortak pakette)	Aynı kurallar hem formda hem API'de çalışır, iki kez yazılmaz.
Önyüz	**React 19 + Vite 6**	Hızlı geliştirme, geniş ekosistem.
Tasarım	**Tailwind CSS 4** + kendi bileşenlerimiz	Modern ve duyarlı (mobil uyumlu) arayüz.
Veri yönetimi	**TanStack Query**	Önbellekleme ve otomatik yenileme; bir randevu değişince ilgili ekranlar kendiliğinden güncellenir.
Formlar	**React Hook Form**	Performanslı form yönetimi; Zod ile bağlanır.
Takvim	**FullCalendar 6**	Ay/hafta/gün/liste görünümleri, sürükle-bırak.
Grafik	**Recharts**	Özet ekranındaki yoğunluk grafiği.
Çok dillilik	**i18next**	Türkçe/İngilizce çeviri altyapısı.
Test	**Vitest**	Hızlı birim ve entegrasyon testleri.

2.2 Mimari: üç paketli tek depo (monorepo)

```
ritma/
├── packages/shared   → Ortak kurallar: Zod şemaları, tipler, randevu durum makinesi
├── apps/api          → Sunucu: iş mantığı, veritabanı, güvenlik, hatırlatma görevi
└── apps/web          → Tarayıcı arayüzü
```

`packages/shared` iki tarafın da kullandığı tek doğruluk kaynağıdır. Örneğin "telefon numarası geçerli mi?" ya da "Tamamlandı durumundaki randevu Bekliyor'a çekilebilir mi?" kuralları burada bir kez yazılır.

2.3 API'nin katmanlı yapısı

Her modül (müşteriler, randevular vb.) aynı üç katmandan oluşur:

```
İstek → routes (HTTP + girdi doğrulama) → service (iş kuralları) → repository (veritabanı) → Prisma
```

- **routes**: Gelen veriyi Zod ile doğrular ve doğru HTTP kodunu döndürür.
- **service**: Kuralların yaşadığı yerdir (çakışma kontrolü, durum geçişleri, telefon benzersizliği).
- **repository**: Yalnızca veritabanı sorgularını içerir.

Bu ayrım sayesinde örneğin veritabanı değişse bile iş kuralları etkilenmez.

2.4 Geliştirme adımları (sırasıyla)

1. **Ortam hazırlığı.** Bilgisayarda Node.js yoktu; `winget` ile Node.js 24 LTS kuruldu.
2. **Ortak paket** (`packages/shared`). Sabitler ve durum makinesi, zaman yardımcıları (aralık çakışması), Türkçe karakter katlama (arama için), tüm Zod şemaları ve API tipleri yazıldı. Hata mesajları doğrudan metin değil **çeviri anahtarıdır** (`validation.phone` gibi); arayüz bunu seçili dile çevirir.
3. **Veritabanı şeması** (`apps/api/prisma/schema.prisma`). Tablolar: `User` , `Customer` , `Service` , `Staff` , `StaffService` (personel–hizmet eşleşmesi), `Appointment` , `NotificationLog` . Önemli tasarım kararları:
   - Ücretler **kuruş cinsinden tam sayı** tutulur (`priceCents`), böylece ondalık yuvarlama hatası olmaz.
   - Randevu oluşturulurken hizmet ücreti randevuya **kopyalanır**; sonradan yapılan zam geçmiş kayıtları değiştirmez.
   - Müşterilerde `searchKey` alanı aramayı Türkçe karakterlere duyarsız yapar ("sukru", "Şükrü"yü bulur).
   - `NotificationLog` tablosundaki benzersiz kısıt, aynı hatırlatmanın iki kez gönderilmesini veritabanı seviyesinde engeller.
4. **Çekirdek katman.** Ayarlar, veritabanı bağlantısı, merkezî hata yönetimi (her hata `{ error: { code, message } }` biçiminde döner) ve oturum doğrulaması yazıldı.
5. **Modüller.** Sırasıyla auth → müşteriler → randevular → hizmetler → personel → bildirimler → raporlar yazıldı.
6. **Çakışma kontrolü.** Randevunun bitiş saati hizmet süresinden hesaplanır. Aynı **personelin** veya aynı **müşterinin** iptal edilmemiş bir randevusuyla zaman aralığı kesişirse kayıt reddedilir. Kontrol ve kayıt tek bir *serializable transaction* içinde yapılır; aynı anda gelen iki istek ikisi birden geçemez.
7. **Hatırlatma sistemi.** `NotificationProvider` arayüzü tanımlandı; şimdilik `MockSmsProvider` ve `MockEmailProvider` mesajları yalnızca kaydediyor. Arka plan görevi her dakika çalışır.
8. **Örnek veri** (`prisma/seed.ts`). Yönetici hesabı, 5 hizmet, 3 personel, 12 müşteri ve son 3 hafta ile önümüzdeki 10 güne yayılmış ~240 randevu eklendi. Veriler deterministiktir, yani her seferinde aynı üretilir.
9. **Arkayüz testleri.** 18 entegrasyon/birim testi ve ortak pakette 7 test yazıldı ve hepsi geçti.
10. **Arayüz.** Sırasıyla temel bileşenler (Button, Dialog, form alanları), çeviri dosyaları, giriş ekranı, menü, ardından her modülün sayfası yazıldı.
11. **Tarayıcıda uçtan uca kontrol.** Gerçek tarayıcıda denenirken bulunan ve düzeltilen hatalar:

- Takvim sayfası sonsuz yeniden çizim döngüsüne giriyordu (FullCalendar seçenek nesneleri her çizimde yeniden oluşuyordu). Sabitlere taşındı.
- Bir tarayıcı eklentisi buton metnini değiştirince yükleniyor simgesi eklenirken uygulama çöküyordu. Buton metni bir `<span>` içine alınarak dayanıklı hale getirildi.
- Hatırlatma metni aynı gün içindeki randevular için de "yarın" diyordu. Artık gerçek tarihe göre "bugün" veya "yarın" yazıyor.
- Beklenmeyen hatalarda teknik döküm yerine kullanıcı dostu bir "Sayfayı yenile" ekranı eklendi.

12. **Üretim modu doğrulandı.** `npm run build` ve ardından `npm start` ile API, derlenmiş arayüzü tek süreçten sunar; giriş ve sayfa yönlendirmesi test edildi.

2.5 Güvenlik önlemleri

Önlem	Açıklama
Şifre saklama	bcrypt ile özetlenir; düz metin şifre veritabanında tutulmaz.
Oturum	JWT, JavaScript'in okuyamadığı `httpOnly` çerezde; `SameSite=Lax` ile siteler arası istek koruması. Üretimde ayrıca `Secure` (yalnızca HTTPS).
Kaba kuvvet koruması	Giriş için dakikada en fazla 10 deneme.
Kullanıcı keşfi	Var olmayan e-postayla giriş de aynı sürede yanıtlanır; hangi hesapların var olduğu anlaşılamaz.
Girdi doğrulama	Her istek Zod ile doğrulanır; Prisma parametreli sorgu kullanır.
Yetki	Giriş ve sağlık kontrolü dışındaki tüm API uçları oturum ister.

---

3. Kurulum

3.1 Gereksinimler

- **Node.js 22 veya üstü** (önerilen: 24 LTS)
  - Windows: `winget install OpenJS.NodeJS.LTS`
  - Kurulumdan sonra terminali kapatıp yeniden açın; `node -v` komutu sürümü göstermeli.
- Ek bir veritabanı kurmaya gerek yoktur (SQLite dosya tabanlıdır).

3.2 Bu bilgisayarda

Kurulum zaten yapıldı. Doğrudan Bölüm 4'e geçebilirsiniz.

3.3 Başka bir bilgisayarda sıfırdan

1. Proje klasörünü kopyalayın (`node_modules` klasörünü kopyalamanıza gerek yok).
2. Ayar dosyasını oluşturun:

   ```
   Copy-Item apps\api\.env.example apps\api\.env
   ```

   `apps\api\.env` dosyasını açıp `JWT_SECRET` değerini **en az 32 karakterlik rastgele bir metinle** değiştirin. PowerShell ile üretmek için:

   ```
   $b = New-Object byte[] 48; [Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b);
   [Convert]::ToBase64String($b)
   ```

3. Kök klasörde kurulumu çalıştırın:

   ```
   npm run setup
   ```

   Bu komut bağımlılıkları kurar, veritabanı tablolarını oluşturur ve örnek verileri yükler.

> **npm 11.19 ve üzeri için not:** npm, paketlerin kurulum betiklerini onaysız çalıştırmaz. Bu projede gerekli onaylar `package.json` içindeki `allowScripts` bölümüne zaten kaydedildi. Yine de Prisma veya esbuild hata verirse:
>
> ```
> npm install-scripts approve @prisma/client prisma @prisma/engines esbuild
> npm rebuild
> ```

---

4. Çalıştırma

4.1 Geliştirme modu (günlük kullanım için de uygundur)

Proje kök klasöründe (`C:\ritma`):

```
npm run dev
```

İki sunucu birlikte başlar:

Sunucu	Adres
Web arayüzü	**http://localhost:5173** ← tarayıcıda bunu açın
API	http://127.0.0.1:3001 (arayüz kendi kullanır)

Durdurmak için terminalde **Ctrl + C**.

Varsayılan giriş bilgileri: `admin@ritma.local` / `admin123`

4.2 Üretim modu

```
npm run build   # arayüzü derler (apps/web/dist)
npm start       # API'yi üretim modunda başlatır; arayüzü de aynı adresten sunar
```

Adres: **http://127.0.0.1:3001**

> ⚠️ Üretim modunda oturum çerezi yalnızca **HTTPS** üzerinden gönderilir. `localhost` / `127.0.0.1` üzerinde tarayıcılar buna izin verir. Ancak uygulamayı ağdaki başka bir cihazdan veya internetten açacaksanız önüne HTTPS sağlayan bir ters vekil sunucu (ör. Caddy, Nginx) koymanız gerekir; aksi halde giriş yapılamaz.

---

5. Kullanım kılavuzu (ekran ekran)

5.1 Giriş ekranı

1. E-posta ve şifrenizi girip **Giriş yap**'a basın.
2. Sağ üstteki **TR / EN** düğmesiyle giriş yapmadan dili değiştirebilirsiniz.
3. Geliştirme modunda alanlar demo hesapla dolu gelir ve altta demo bilgisi görünür. Üretim modunda bu gösterilmez.
4. Hatalı şifrede "E-posta veya şifre hatalı" uyarısı çıkar. Çok sayıda hatalı denemede bir dakika beklemeniz istenir.

5.2 Menü ve dil

- Sol menü: **Özet, Takvim, Müşteriler, Hizmetler, Personel, Bildirimler**.
- Mobilde menü sol üstteki ☰ simgesiyle açılır.
- Menünün altında **TR / EN** dil düğmesi, kullanıcı adınız ve **çıkış** simgesi bulunur.
- Dil seçimi tarayıcıda saklanır; bir sonraki girişte aynı dille açılır. Tarih, saat ve para biçimleri de seçilen dile göre değişir.

5.3 Özet (Dashboard)

Sağ üstten dönem seçin: **Bugün / Bu hafta / Bu ay**.

Kart	Anlamı
Toplam randevu	Dönemdeki tüm randevular; altında aktif (bekleyen + onaylı) sayısı.
Tamamlanan işlem	Tamamlandı durumundaki randevular; altında tamamlanma oranı (tamamlanan ÷ tamamlanan + iptal).
Gerçekleşen gelir	Tamamlanan randevuların ücret toplamı.
Beklenen gelir	Bekleyen ve onaylı randevulardan gelecek tahmini gelir.

- **Yoğunluk grafiği**: Haftalık ve aylık görünümde gün gün, "Bugün" görünümünde saat saat randevu sayısını durumlara göre (tamamlandı / aktif / iptal) üst üste gösterir. Çubuğun üzerine gelince ayrıntı çıkar. Sağ üstteki simgeyle **tablo görünümüne** geçebilirsiniz.
- **Hizmet bazında**: En çok tercih edilen hizmetler ve her birinden elde edilen gelir.
- **Yaklaşan randevular**: Sıradaki 6 aktif randevu. Müşteri adına tıklayınca profili açılır.

5.4 Takvim

Görünümler: Ay · Hafta · Gün · Ajanda (liste). Mobilde varsayılan görünüm Ajanda'dır.

Renkler ve durumlar:

Durum	Görünüm
Bekliyor	Sarı
Onaylandı	Mor
Tamamlandı	Yeşil
İptal Edildi	Gri, üstü çizili

Yeni randevu oluşturma:

1. Takvimde boş bir saate tıklayın (başlangıç saati otomatik dolar) **veya** sağ üstteki **+ Yeni randevu** düğmesine basın.
2. **Müşteri**: Ad veya telefon yazmaya başlayın, listeden seçin. Ok tuşları ve Enter ile de seçilebilir.
3. **Hizmet**: Listeden seçin (süre ve ücret yanında yazar).
4. **Personel** (isteğe bağlı): Yalnızca o hizmeti veren aktif personel listelenir. "Fark etmez" seçilirse randevu personelsiz kaydedilir.
5. **Tarih** ve **Başlangıç saati**'ni seçin. Altta mavi bir özet satırı bitiş saatini, süreyi ve ücreti gösterir.
6. İsterseniz not ekleyip **Kaydet**'e basın. Yeni randevu **Bekliyor** durumunda oluşur.

Çakışma kontrolü: Kaydederken sistem şu durumlarda randevuyu **reddeder**:

- Seçilen **personelin** aynı saat aralığında başka bir randevusu varsa: "Seçilen personelin bu saat aralığında başka bir randevusu var."
- **Müşterinin** aynı saat aralığında başka bir randevusu varsa (personel farklı olsa bile).

Hata mesajının altında çakışan randevunun saati de yazar. Arka arkaya randevular (ör. 10:00–11:00 ve 11:00–12:00) çakışma sayılmaz. İptal edilmiş randevular saati boşaltır.

Randevuyu açma ve düzenleme: Takvimdeki randevuya tıklayın. Açılan pencerede:

- Üstte mevcut durum ve **Müşteri profili** bağlantısı.
- **Durum düğmeleri** (yalnızca izin verilen geçişler görünür):
  - Bekliyor → **Onayla**, **Tamamlandı**, **İptal et**
  - Onaylandı → **Beklemeye al**, **Tamamlandı**, **İptal et**
  - **Tamamlandı** ve **İptal Edildi** kesin durumlardır, geri alınamaz. Bu yüzden bu ikisi için onay penceresi çıkar.
- Alttaki formda müşteri, hizmet, personel, tarih/saat ve not değiştirilebilir. Tamamlanan ve iptal edilen randevular yalnızca görüntülenir.

Sürükle-bırak ile taşıma: Hafta veya Gün görünümünde aktif bir randevuyu tutup başka bir saate sürükleyin. Yeni saat çakışıyorsa randevu otomatik olarak eski yerine döner ve uyarı gösterilir. Süre sürüklenerek değiştirilemez; süre hizmetten gelir.

Personel filtresi: Sağ üstteki "Tüm personel" listesinden bir kişi seçerek yalnızca onun randevularını görebilirsiniz.

> Randevunun saati değiştirildiğinde o randevu için gönderilmiş hatırlatmalar sıfırlanır; müşteri yeni saat için tekrar hatırlatma alır.

5.5 Müşteriler

Liste ekranı:

- Üstteki arama kutusu ad, telefon veya e-postada arar. Türkçe karakter ve büyük/küçük harf farkı gözetmez ("isik", "Işık"ı bulur).
- Tabloda telefon, e-posta, randevu sayısı ve son randevu tarihi görünür. Dar ekranlarda bazı sütunlar gizlenir.
- Satıra tıklayınca müşteri profili açılır. 🖍️ düzenler, 🗑️ siler.
- Sayfa başına 15 kayıt gösterilir; altta sayfa geçişi bulunur.

Yeni müşteri (sağ üst düğme):

Alan	Kural
Ad Soyad	Zorunlu, en az 2 karakter
Telefon	Zorunlu ve **benzersiz**. Boşluk, tire ve parantezler otomatik temizlenir ("0 (532) 111-22 33" → "05321112233").
E-posta	İsteğe bağlı; girilirse geçerli olmalı. Hatırlatma e-postaları buraya gider.
Özel notlar	İsteğe bağlı (tercihler, alerjiler vb.)

Müşteri profili:

- Üstte **Randevu oluştur** (müşteri otomatik seçili gelir), **Düzenle**, **Sil** düğmeleri.
- İstatistik kartları: toplam randevu, tamamlanan, yaklaşan, toplam harcama (yalnızca tamamlanan randevular).
- İletişim bilgileri (telefona veya e-postaya tıklayınca arama veya e-posta başlatılır) ve özel notlar.
- **Randevu ve işlem geçmişi**: En yeniden eskiye tüm randevular. Bir satıra tıklayınca randevu penceresi açılır.

> ⚠️ Müşteri silindiğinde **tüm randevu geçmişi de kalıcı olarak silinir**. Bu işlem geri alınamaz.

5.6 Hizmetler

- **Yeni hizmet**: Ad, süre (dakika, 5–600), ücret (₺, kuruşlu girilebilir) ve "Aktif" seçeneği.
- Kartlarda süre, o hizmeti veren aktif personel sayısı ve ücret görünür.
- **Pasif** hizmetler yeni randevularda listelenmez, ancak mevcut randevular etkilenmez.
- Ücret değiştirildiğinde **yalnızca yeni randevular** yeni ücretle oluşur; geçmiş randevuların ücreti korunur.

- Randevusu olan bir hizmet **silinemez**. Bunun yerine pasif yapın.

5.7 Personel

- **Yeni personel**: Ad Soyad (zorunlu), telefon ve e-posta (isteğe bağlı), **verdiği hizmetler** (onay kutuları), "Aktif" seçeneği.
- Randevu formunda bir personel yalnızca işaretlediğiniz hizmetler için listelenir.
- Pasif personel yeni randevulara atanamaz.
- Randevusu olan personel **silinemez**. Bunun yerine pasif yapın.

5.8 Bildirimler

- Sistem **her dakika** otomatik olarak çalışır ve şunları gönderir:
  - Randevuya **24 saatten az** kalmışsa: "yarın / bugün … randevunuz bulunmaktadır" hatırlatması
  - Randevuya **1 saatten az** kalmışsa: "yaklaşık 1 saat sonra" hatırlatması
- **SMS** müşterinin telefonuna, **e-posta** (adres kayıtlıysa) e-postasına gönderilir.
- Yalnızca **Bekliyor** ve **Onaylandı** durumundaki randevular için gönderilir. Her hatırlatma kanal başına **en fazla bir kez** gider.
- Kısa süre önce alınan bir randevu, yalnızca anlamlı olan hatırlatmayı alır (ör. 30 dakika sonraki randevu yalnızca "1 saat" hatırlatmasını alır).
- **Şimdi tara** düğmesi bir dakikalık bekleme süresini beklemeden taramayı hemen yapar.

> ℹ️ **Şu an mesajlar gerçekten gönderilmez.** Mock (simülasyon) sağlayıcılar mesajı bu sayfaya ve sunucu günlüğüne yazar. Gerçek SMS/e-posta için bkz. Bölüm 11.

---

6. İlk gün senaryosu (adım adım)

Örnek verileri temizleyip kendi işletmenizi baştan kurmak için:

1. **Kendi yönetici bilgilerinizi belirleyin.** `apps\api\.env` içinde `ADMIN_EMAIL`, `ADMIN_PASSWORD`, `ADMIN_NAME` ve `BUSINESS_NAME` değerlerini düzenleyin.
2. **Veritabanını sıfırlayın** (⚠️ tüm veriler silinir):

   ```
   npm run db:reset -w @ritma/api
   ```

   Bu komut örnek verileri de yeniden yükler. Örnek müşteri ve randevuları istemiyorsanız aşağıdaki "Örnek veri olmadan başlamak" notuna bakın.
3. `npm run dev` ile başlatın ve yeni bilgilerinizle giriş yapın.
4. **Hizmetler** → hizmetlerinizi ekleyin (ad, süre, ücret).
5. **Personel** → çalışanlarınızı ekleyin ve her birinin verdiği hizmetleri işaretleyin.
6. **Müşteriler** → müşterilerinizi ekleyin (veya randevu alırken ekleyin).
7. **Takvim** → boş bir saate tıklayıp ilk randevunuzu oluşturun.
8. Müşteri randevuyu teyit edince randevuyu açıp **Onayla**'ya basın.
9. Hizmet verildikten sonra randevuyu **Tamamlandı** yapın. Gelir artık **Özet** ekranındaki "Gerçekleşen gelir"e yansır.
10. Gelmeyen veya vazgeçen müşteri için **İptal et** kullanın; o saat başka randevuya açılır.

> **Örnek veri olmadan başlamak:** `apps\api\prisma\seed.ts` dosyasındaki `main()` fonksiyonunda yönetici hesabını oluşturan `prisma.user.upsert(...)` bloğundan sonraki hizmet/personel/müşteri/randevu ekleme kısımlarını kaldırın, sonra 2. adımı uygulayın.

7. Yapılandırma (.env)

Dosya: `apps\api\.env` (örnek dosya: `.env.example`). Değişiklikten sonra sunucuyu yeniden başlatın.

Değişken	Varsayılan	Açıklama
`DATABASE_URL`	`file:./dev.db`	Veritabanı adresi. SQLite dosyası `apps/api/prisma/dev.db` konumundadır.
`JWT_SECRET`	*(rastgele üretildi)*	Oturum imza anahtarı. **Üretimde zorunlu, en az 32 karakter.** Değiştirilirse tüm oturumlar kapanır.
`SESSION_HOURS`	`12`	Oturum süresi (saat).
`PORT` / `HOST`	`3001` / `127.0.0.1`	API'nin dinlediği adres.
`REMINDERS_ENABLED`	`true`	`false` yapılırsa otomatik hatırlatma görevi çalışmaz.
`REMINDER_INTERVAL_MS`	`60000`	Hatırlatma tarama sıklığı (milisaniye).
`NOTIFY_LOCALE`	`tr`	Müşteriye giden mesajların dili (`tr` / `en`).
`BUSINESS_NAME`	`Ritma`	Mesajların sonunda görünen işletme adı.
`BUSINESS_TIMEZONE`	`Europe/Istanbul`	Mesajlardaki tarih/saat bu saat dilimine göre yazılır.
`ADMIN_EMAIL` / `ADMIN_PASSWORD` / `ADMIN_NAME`	`admin@ritma.local` / `admin123` / `Yönetici`	Seed (örnek veri) betiğinin oluşturduğu yönetici.

Arayüz için isteğe bağlı ayar: `apps\web\.env` içinde `VITE_CURRENCY=TRY` (ör. `EUR`, `USD`). Bu ayar yalnızca para biriminin gösterimini değiştirir; çeviri yapmaz.

8. Geliştirici komutları ve testler

Tüm komutlar proje kök klasöründe çalıştırılır.

Komut	Ne yapar
`npm run setup`	İlk kurulum (bağımlılıklar + migration + örnek veri)
`npm run dev`	Geliştirme sunucuları (API + web, kod değişince otomatik yenilenir)
`npm run build`	Arayüzü üretim için derler
`npm start`	Üretim modunda başlatır
`npm test`	Tüm testleri çalıştırır
`npm run typecheck`	Tüm paketlerde TypeScript kontrolü
`npm run db:reset -w @ritma/api`	Veritabanını sıfırlar + örnek veri (⚠️ veri siler)
`npm run db:migrate -w @ritma/api`	Şema değişikliğinden sonra yeni migration oluşturur
`npm run db:studio -w @ritma/api`	Prisma Studio: verileri tarayıcıda tablo olarak gösterir

Testlerin kapsadıkları (toplam 25 test):

- Çakışma: aynı personel, aynı müşteri, arka arkaya randevular, iptalin saati boşaltması, kendi saatini değiştirme
- Durum geçişleri ve düzenlenemeyen randevular
- Personelin vermediği hizmete atanamaması
- Telefon benzersizliği, Türkçe karakterli arama, müşteri istatistikleri
- Randevusu olan hizmetin silinememesi
- Yetkisiz erişim ve hatalı şifre
- Hatırlatma pencereleri, tekrar gönderilmemesi, iletişim bilgisi olmayan kanalın atlanması, "bugün/yarın" metni

Testler ayrı bir veritabanı (`prisma/test.db`) kullanır; gerçek verilerinize dokunmaz.

Veritabanı şemasını değiştirmek (ör. müşteriye doğum tarihi eklemek):

1. `apps/api/prisma/schema.prisma` içinde alanı ekleyin.
2. `npm run db:migrate -w @ritma/api` komutunu çalıştırıp migration'a bir ad verin.
3. `packages/shared` içindeki şemayı ve tipi, ardından API ve formu güncelleyin.

PostgreSQL'e geçiş:

1. `schema.prisma` → `provider = "postgresql"`
2. `.env` → `DATABASE_URL="postgresql://kullanici:sifre@localhost:5432/ritma"`

3. `apps/api/prisma/migrations` klasörünü silip `npm run db:migrate -w @ritma/api` ile PostgreSQL için yeniden oluşturun. Mevcut migration'lar SQLite'a özeldir.

---

9. Klasör yapısı

```
ritma/
├── package.json                Kök: workspace tanımı ve ortak komutlar
├── .gitignore, .gitattributes  Git ayarları (depoya girmeyen dosyalar, satır sonları)
├── README.md                   Kısa teknik özet
├── docs/KULLANIM-KILAVUZU.md   Bu belge (düzenlenebilir hali)
├── docs/KULLANIM-KILAVUZU.pdf  Bu belgenin PDF hali
│
├── packages/shared/src/
│   ├── constants.ts            Randevu durumları ve izin verilen geçişler
│   ├── time.ts                 Zaman aralığı çakışması
│   ├── text.ts                 Türkçe karakter katlama, telefon temizleme
│   ├── types.ts                API yanıt tipleri
│   └── schemas/                Zod doğrulama şemaları (müşteri, hizmet, personel, randevu, rapor,
giriş)
│
├── apps/api/
│   ├── .env                    Ayarlar (gizli; paylaşmayın)
│   ├── prisma/
│   │   ├── schema.prisma       Veritabanı şeması
│   │   ├── migrations/         Şema geçmişi
│   │   ├── seed.ts             Örnek veri
│   │   └── dev.db              SQLite veritabanı dosyası (yedeklemek için bunu kopyalayın)
│   ├── src/
│   │   ├── server.ts           Başlangıç noktası
│   │   ├── app.ts              Eklentiler, rotalar, hatırlatma görevi
│   │   ├── core/               Ayarlar, DB, hata yönetimi, oturum doğrulama
│   │   └── modules/
│   │       ├── auth/           Giriş / çıkış / oturum
│   │       ├── customers/      Müşteri CRUD, arama, geçmiş
│   │       ├── appointments/   Randevu + çakışma kontrolü + durum değişimi
│   │       ├── services/       Hizmetler
│   │       ├── staff/          Personel
│   │       ├── notifications/  Sağlayıcılar, şablonlar, hatırlatma kuralı ve görevi
│   │       └── reports/        Özet ekranı hesapları
│   └── test/                   API testleri
│
└── apps/web/src/
    ├── main.tsx, router.tsx    Uygulama girişi ve sayfa yönlendirme
    ├── i18n/locales/           tr.ts ve en.ts çeviri dosyaları
    ├── lib/                    API istemcisi, biçimlendirme, yardımcılar
    ├── components/ui/          Button, Dialog, form alanları, kartlar
    ├── components/layout/      Menü, sayfa başlığı, dil seçici, hata ekranı
    └── features/               auth · dashboard · appointments · customers · services · staff ·
notifications
```

> **Yedekleme:** SQLite kullanırken tüm veriler `apps\api\prisma\dev.db` dosyasındadır. Sunucu kapalıyken bu dosyayı kopyalamak yedek almak için yeterlidir.

10. Sorun giderme

Belirti	Çözüm
`node` / `npm` tanınmıyor	Node.js kurulumundan sonra terminali kapatıp açın.
`EADDRINUSE` (port kullanımda)	Başka bir `npm run dev` açık kalmış olabilir. O terminali kapatın veya `.env` içinde `PORT` değerini değiştirin. Web portu 5173'tür.
Prisma veya esbuild hatası, "install scripts" uyarısı	Bölüm 3.3 sonundaki `npm install-scripts approve …` adımı.
Arayüz açılıyor ama "Sunucuya ulaşılamıyor" uyarısı veriyor	API çalışmıyordur. `npm run dev` çıktısındaki `[api]` satırlarında hata olup olmadığına bakın.
Giriş sonrası sürekli giriş ekranına dönüyor	Üretim modunu HTTP üzerinden başka bir cihazdan açıyorsanız HTTPS gerekir (bkz. 4.2).
"Beklenmeyen bir hata oluştu" ekranı	**Sayfayı yenile**'ye basın. Sık tekrarlıyorsa çeviri ve parola yöneticisi gibi sayfayı değiştiren tarayıcı eklentilerini bu site için kapatmayı deneyin.
Giriş butonuna ilk basışta bir şey olmuyor	Chrome'un otomatik doldurma penceresi açık olabilir. Önce boş bir yere tıklayıp sonra butona basın.
Hatırlatma gitmiyor	Randevunun Bekliyor veya Onaylandı durumunda ve 24 saatten yakın olduğundan emin olun. E-posta için müşteride e-posta adresi kayıtlı olmalı. `REMINDERS_ENABLED=true` olmalı.
Yönetici şifresini unuttum	Şu an şifre değiştirme ekranı yok. `.env` içindeki `ADMIN_PASSWORD` yalnızca **sıfırlamada** kullanılır (`db:reset` tüm veriyi siler). Bkz. Bölüm 11.
Testler çalışırken "Prisma Migrate detected that it was invoked by…" hatası	Yapay zekâ asistanlarına özgü bir Prisma korumasıdır; elle çalıştırdığınızda görülmez.

11. Bilinen sınırlamalar ve sonraki adımlar

Bu sürümde **olmayan** ve ileride eklenebilecek özellikler:

Konu	Mevcut durum	Öneri
Gerçek SMS / e-posta	Mock sağlayıcılar yalnızca kayıt tutar.	`NotificationProvider` arayüzünü uygulayan bir sınıf yazın (ör. Netgsm/İletimerkezi için SMS, SMTP/SES için e-posta) ve `apps/api/src/app.ts` içinde mock sağlayıcıyla değiştirin.
Kullanıcı yönetimi	Tek yönetici hesabı; şifre değiştirme ekranı yok.	Kullanıcı ekleme, roller (yönetici/personel) ve şifre değiştirme ekranı.
Çalışma saatleri	Herhangi bir saate randevu verilebilir; takvim 08:00–21:00 aralığını gösterir.	İşletme ve personel çalışma saatleri, tatil günleri kontrolü.
Geçmiş tarih	Geçmiş tarihe randevu girilebilir (sonradan kayıt için).	İstenirse engellenebilir.
Müşterinin kendi randevusunu alması	Yok.	Herkese açık online randevu sayfası.
Karanlık tema	Yok.	Tailwind ile eklenebilir.
Toplu dışa aktarma	Yok.	Müşteri listesi ve raporlar için Excel/CSV dışa aktarma.
Büyük veri	Raporlar seçilen dönemi bellekte hesaplar; küçük ve orta ölçekli işletme için yeterli.	Çok yüksek hacimde veritabanı tarafında gruplama (SQL `GROUP BY`).

12. Git ve GitHub

Proje Git ile sürüm takibi altındadır ve GitHub'da **herkese açık** bir depoda durur:

https://github.com/tgcksr/ritma

12.1 Neler kuruldu ve ayarlandı

Öğe	Açıklama
Git (MinGit)	Git'in hafif, komut satırı sürümü. Normal Git kurulumu yönetici izni istediği için bu sürüm seçildi. Komut satırında tam çalışır; "Git Bash" uygulaması ve sağ tık menüsü kısayolları yoktur.
GitHub CLI (`gh`)	GitHub hesabına giriş ve depo oluşturma aracı. Giriş bu bilgisayarda kayıtlıdır; tekrar giriş yapmanız gerekmez.
Commit yazarı	`tgcksr`. E-posta olarak GitHub'ın gizli adresi (`…@users.noreply.github.com`) kullanılır, böylece gerçek e-postanız herkese açık görünmez. Bu ayar yalnızca bu proje için yapıldı.
`.gitignore`	Depoya **girmeyen** dosyalar: `.env` (gizli anahtar), `*.db` (veritabanı), `node_modules`, `dist` (derleme çıktısı), `.claude/` ve `CLAUDE.md` (bilgisayara özel asistan dosyaları).
`.gitattributes`	Satır sonlarını sabitler; dosyalar Windows ve diğer sistemlerde aynı kalır.

12.2 Değişiklikleri GitHub'a göndermek

Kodda veya belgelerde değişiklik yaptıktan sonra proje klasöründe:

```
cd C:\ritma
git status                              # neyin değiştiğini görün
git add -A                              # tüm değişiklikleri hazırlayın
git commit -m "Ne değiştiğini kısaca yazın"
git push                                # GitHub'a gönderin
```

Sık kullanılan diğer komutlar:

Komut	Ne yapar
`git log --oneline`	Geçmiş commit'leri listeler
`git diff`	Henüz commit'lenmemiş değişiklikleri gösterir
`git restore <dosya>`	Bir dosyadaki kaydedilmemiş değişiklikleri geri alır (⚠️ değişiklik kaybolur)
`git pull`	GitHub'daki güncel hali bilgisayara çeker (başka bir bilgisayarda değişiklik yaptıysanız)

12.3 Projeyi başka bir bilgisayara indirmek

```
git clone https://github.com/tgcksr/ritma.git
cd ritma
Copy-Item apps\api\.env.example apps\api\.env   # ardından JWT_SECRET'ı değiştirin (Bölüm 3.3)
npm run setup
npm run dev
```

`.env` ve veritabanı GitHub'da olmadığı için yeni bilgisayarda örnek verilerle boş bir kurulum oluşur. Mevcut verilerinizi taşımak için `apps\api\prisma\dev.db` dosyasını elle kopyalayın.

12.4 Güvenlik notları

- Depo **herkese açık**: Kodu herkes görebilir. Gizli bilgiler (`.env`) depoya girmez; bunu bozmamak için `.gitignore` dosyasından bu satırları silmeyin.
- Demo giriş bilgileri (`admin@ritma.local` / `admin123`) README'de ve bu kılavuzda yazılıdır. Uygulamayı internete açacaksanız şifreyi mutlaka değiştirin (Bölüm 6).
- GitHub CLI'a verilen yetkiyi istediğiniz zaman GitHub'da **Settings → Applications → Authorized OAuth Apps → GitHub CLI** bölümünden kaldırabilirsiniz.
- Depoyu gizli yapmak için: GitHub'da depo sayfası → **Settings → General → Danger Zone → Change repository visibility**.

12.5 Tam Git kurulumu (isteğe bağlı)

Git Bash ve sağ tık menüsü kısayollarını isterseniz, yönetici izni penceresinde **Evet** diyerek tam sürümü kurabilirsiniz:

```
winget install --id Git.Git -e
```